

SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(SIMATS)
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

LAB MANUAL (PART 1)
CSA37 - SOFTWARE TESTING
EXPERIMENTS 16 TO 25
ACADEMIC YEAR: 2025 - 2026

EXPERIMENT 16

JUNIT PROGRAM TO CHECK ANAGRAM STRINGS

AIM

To check whether a string is an anagram of another string and validate output using JUnit assertions.

ALGORITHM / PROCEDURE

1. Read two strings s1 and s2.
2. Remove spaces and convert to lowercase.
3. Convert strings to character arrays.
4. Sort both arrays.
5. Compare both arrays using Arrays.equals().
6. Return true if equal else false.

PROGRAM / CODE

```
import java.util.Arrays;
import org.junit.Test;
import static org.junit.Assert.*;

public class AnagramTest {

    public boolean isAnagram(String s1, String s2) {

        if (s1 == null || s2 == null)
            return false;

        s1 = s1.replaceAll("\\s", "").toLowerCase();
        s2 = s2.replaceAll("\\s", "").toLowerCase();

        if (s1.length() != s2.length())
            return false;
```

```

        char[] a1 = s1.toCharArray();
        char[] a2 = s2.toCharArray();

        Arrays.sort(a1);
        Arrays.sort(a2);

        return Arrays.equals(a1, a2);
    }

    @Test
    public void testAnagramTrue() {
        assertTrue(isAnagram("listen", "silent"));
    }

    @Test
    public void testAnagramFalse() {
        assertFalse(isAnagram("hello", "world"));
    }

    @Test
    public void testAnagramWithSpaces() {
        assertTrue(isAnagram("Dirty room", "Dormitory"));
    }
}

```

WHITE BOX TEST CASES

Test Case	Input	Expected Output
TC1	listen, silent	True
TC2	hello, world	False
TC3	Dirty room, Dormitory	True

INPUT

listen, silent

OUTPUT

True

RESULT

Thus, anagram checking was executed successfully and validated using JUnit.

EXPERIMENT 17

JUNIT PROGRAM TO FIND MAXIMUM OF THREE NUMBERS

AIM

To find the maximum of three numbers and validate output using JUnit assertions.

ALGORITHM / PROCEDURE

1. Read three integers a, b, c.
2. Initialize max=a.
3. If $b > \text{max}$, set max=b.
4. If $c > \text{max}$, set max=c.
5. Return max.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;

public class MaximumOfThreeNumbersTest {

    public int findMax(int a, int b, int c) {

        int max = a;

        if (b > max)
            max = b;

        if (c > max)
            max = c;

        return max;
    }

    @Test
```

```

public void testMaxNormal() {
    assertEquals(30, findMax(10, 20, 30));
}

@Test
public void testMaxNegative() {
    assertEquals(-5, findMax(-10, -5, -20));
}

@Test
public void testMaxSameValues() {
    assertEquals(15, findMax(15, 15, 15));
}
}

```

WHITE BOX TEST CASES

Test Case	Input	Expected Output
TC1	10,20,30	30
TC2	-10,-5,-20	-5
TC3	15,15,15	15

INPUT

10,20,30

OUTPUT

30

RESULT

Thus, maximum of three numbers was executed successfully and validated using JUnit.

EXPERIMENT 18

JUNIT PROGRAM TO CHECK EVEN OR ODD NUMBER

AIM

To check whether a number is even or odd and validate output using JUnit assertions.

ALGORITHM / PROCEDURE

1. Read integer n.
2. If $n \% 2 == 0$ return Even.
3. Else return Odd.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;

public class EvenOddTest {

    public String checkEvenOdd(int n) {

        if (n % 2 == 0)
            return "Even";
        else
            return "Odd";
    }

    @Test
    public void testEvenNumber() {
        assertEquals("Even", checkEvenOdd(10));
    }

    @Test
    public void testOddNumber() {
        assertEquals("Odd", checkEvenOdd(15));
    }
}
```

```
@Test
public void testZero() {
    assertEquals("Even", checkEvenOdd(0));
}
}
```

WHITE BOX TEST CASES

Test Case	Input	Expected Output
TC1	10	Even
TC2	15	Odd
TC3	0	Even

INPUT

15

OUTPUT

Odd

RESULT

Thus, even or odd checking was executed successfully and validated using JUnit.

EXPERIMENT 19

COMPOUND INTEREST CALCULATION USING JUNIT

AIM

To calculate compound interest and validate output using JUnit assertions.

ALGORITHM / PROCEDURE

1. Read principal P, rate R, time T.
2. Calculate amount $A = P(1+R/100)^T$.
3. Calculate $CI = A - P$.
4. Return CI.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;

public class CompoundInterestTest {

    public double calculateCompoundInterest(double p, double r, int t) {

        double amount = p * Math.pow((1 + r / 100), t);
        return amount - p;
    }

    @Test
    public void testCompoundInterest() {
        assertEquals(210.0, calculateCompoundInterest(1000, 10, 2), 0.01);
    }

    @Test
    public void testCompoundInterestOneYear() {
        assertEquals(100.0, calculateCompoundInterest(1000, 10, 1), 0.01);
    }
}
```

```
@Test
public void testCompoundInterestZeroRate() {
    assertEquals(0.0, calculateCompoundInterest(1000, 0, 5), 0.01);
}
}
```

WHITE BOX TEST CASES

Test Case	Input	Expected Output
TC1	P=1000,R=10,T=2	210
TC2	P=1000,R=10,T=1	100
TC3	P=1000,R=0,T=5	0

INPUT

P=1000, R=10, T=2

OUTPUT

CI=210

RESULT

Thus, compound interest calculation was executed successfully and validated using JUnit.

EXPERIMENT 20

COUNT VOWELS AND CONSONANTS USING JUNIT

AIM

To count the number of vowels and consonants in a given string and validate output using JUnit assertions.

ALGORITHM / PROCEDURE

1. Read string s.
2. Initialize vowels=0, consonants=0.
3. Traverse each character.
4. If character is vowel increment vowels.
5. Else if alphabet increment consonants.
6. Return count values.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;

public class VowelConsonantCountTest {

    public int[] countVowelsConsonants(String s) {

        int vowels = 0;
        int consonants = 0;

        s = s.toLowerCase();

        for (int i = 0; i < s.length(); i++) {

            char ch = s.charAt(i);
```

```

        if (ch >= 'a' && ch <= 'z') {

            if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u')
                vowels++;
            else
                consonants++;
        }
    }

    return new int[]{vowels, consonants};
}

@Test
public void testVowelConsonantCount() {
    int[] result = countVowelsConsonants("hello");
    assertEquals(2, result[0]);
    assertEquals(3, result[1]);
}

@Test
public void testOnlyVowels() {
    int[] result = countVowelsConsonants("aeiou");
    assertEquals(5, result[0]);
    assertEquals(0, result[1]);
}

@Test
public void testOnlyConsonants() {
    int[] result = countVowelsConsonants("bcdf");
    assertEquals(0, result[0]);
    assertEquals(4, result[1]);
}
}

```

WHITE BOX TEST CASES

Test Case	Input	Expected Output
TC1	hello	V=2,C=3
TC2	aeiou	V=5,C=0
TC3	bcdf	V=0,C=4

INPUT

hello

OUTPUT

Vowels=2, Consonants=3

RESULT

Thus, vowels and consonants counting was executed successfully and validated using JUnit.

EXPERIMENT 21

PRIME NUMBER CHECKING USING JUNIT

AIM

To check whether a given number is prime and validate output using JUnit assertions.

ALGORITHM / PROCEDURE

1. Read integer n.
2. If $n \leq 1$ return false.
3. Loop i from 2 to $\sqrt{n}$.
4. If $n \% i == 0$ return false.
5. Return true.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;

public class PrimeNumberTest {

    public boolean isPrime(int n) {

        if (n <= 1)
            return false;

        for (int i = 2; i <= Math.sqrt(n); i++) {

            if (n % i == 0)
                return false;
        }

        return true;
    }
}
```

```

@Test
public void testPrimeTrue() {
    assertTrue(isPrime(13));
}

@Test
public void testPrimeFalse() {
    assertFalse(isPrime(12));
}

@Test
public void testPrimeOne() {
    assertFalse(isPrime(1));
}
}

```

WHITE BOX TEST CASES

Test Case	Input	Expected Output
TC1	13	True
TC2	12	False
TC3	1	False

INPUT

13

OUTPUT

True

RESULT

Thus, prime number checking was executed successfully and validated using JUnit.

EXPERIMENT 22

REVERSE A NUMBER USING JUNIT

AIM

To reverse a given number and validate output using JUnit assertions.

ALGORITHM / PROCEDURE

1. Read integer n.
2. Initialize rev=0.
3. While n != 0: digit=n%10, rev=rev*10+digit, n=n/10.
4. Return rev.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;

public class ReverseNumberTest {

    public int reverseNumber(int n) {

        int rev = 0;

        while (n != 0) {
            int digit = n % 10;
            rev = rev * 10 + digit;
            n = n / 10;
        }

        return rev;
    }

    @Test
    public void testReverseNormal() {
        assertEquals(321, reverseNumber(123));
    }
}
```

```

    }

    @Test
    public void testReverseWithZero() {
        assertEquals(21, reverseNumber(120));
    }

    @Test
    public void testReverseSingleDigit() {
        assertEquals(7, reverseNumber(7));
    }
}

```

WHITE BOX TEST CASES

Test Case	Input	Expected Output
TC1	123	321
TC2	120	21
TC3	7	7

INPUT

123

OUTPUT

321

RESULT

Thus, reverse number program was executed successfully and validated using JUnit.

EXPERIMENT 23

SUM OF DIGITS OF A NUMBER USING JUNIT

AIM

To find the sum of digits of a given number and validate output using JUnit assertions.

ALGORITHM / PROCEDURE

1. Read integer n.
2. Initialize sum=0.
3. While n != 0: digit=n%10, sum=sum+digit, n=n/10.
4. Return sum.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;

public class SumOfDigitsTest {

    public int sumOfDigits(int n) {

        int sum = 0;

        while (n != 0) {
            int digit = n % 10;
            sum = sum + digit;
            n = n / 10;
        }

        return sum;
    }

    @Test
    public void testSumDigitsNormal() {
        assertEquals(6, sumOfDigits(123));
    }
}
```

```

    }

    @Test
    public void testSumDigitsZero() {
        assertEquals(0, sumOfDigits(0));
    }

    @Test
    public void testSumDigitsLarge() {
        assertEquals(15, sumOfDigits(555));
    }
}

```

WHITE BOX TEST CASES

Test Case	Input	Expected Output
TC1	123	6
TC2	0	0
TC3	555	15

INPUT

123

OUTPUT

6

RESULT

Thus, sum of digits program was executed successfully and validated using JUnit.

EXPERIMENT 24

STRING EQUALITY IGNORING CASE USING JUNIT

AIM

To check whether two strings are equal ignoring case sensitivity and validate output using JUnit assertions.

ALGORITHM / PROCEDURE

1. Read two strings s1 and s2.
2. Convert both strings to lowercase.
3. Compare using equals().
4. Return true if equal else false.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;

public class StringIgnoreCaseTest {

    public boolean compareIgnoreCase(String s1, String s2) {

        if (s1 == null || s2 == null)
            return false;

        return s1.equalsIgnoreCase(s2);
    }

    @Test
    public void testEqualIgnoreCase() {
        assertTrue(compareIgnoreCase("Java", "java"));
    }

    @Test
    public void testNotEqualIgnoreCase() {
```

```

        assertFalse(compareIgnoreCase("Java", "Python"));
    }

    @Test
    public void testNullCase() {
        assertFalse(compareIgnoreCase(null, "test"));
    }
}

```

WHITE BOX TEST CASES

Test Case	Input	Expected Output
TC1	Java, java	True
TC2	Java, Python	False
TC3	null, test	False

INPUT

Java, java

OUTPUT

True

RESULT

Thus, string comparison ignoring case was executed successfully and validated using JUnit.

EXPERIMENT 25

STUDENT GRADE CALCULATION USING JUNIT

AIM

To calculate grade of a student based on marks and validate output using JUnit assertions.

ALGORITHM / PROCEDURE

1. Read marks.
2. If marks ≥ 90 grade A.
3. Else if marks ≥ 75 grade B.
4. Else if marks ≥ 50 grade C.
5. Else grade F.
6. Return grade.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;

public class StudentGradeTest {

    public String calculateGrade(int marks) {

        if (marks >= 90)
            return "A";
        else if (marks >= 75)
            return "B";
        else if (marks >= 50)
            return "C";
        else
            return "F";
    }

    @Test
```

```

public void testGradeA() {
    assertEquals("A", calculateGrade(95));
}

@Test
public void testGradeB() {
    assertEquals("B", calculateGrade(80));
}

@Test
public void testGradeFail() {
    assertEquals("F", calculateGrade(40));
}
}

```

WHITE BOX TEST CASES

Test Case	Input	Expected Output
TC1	95	A
TC2	80	B
TC3	40	F

INPUT

80

OUTPUT

B

RESULT

Thus, student grade calculation was executed successfully and validated using JUnit.

LAB MANUAL (PART 2)
CSA37 - SOFTWARE TESTING
EXPERIMENTS 26 TO 35
ACADEMIC YEAR: 2025 - 2026

EXPERIMENT 26

FIND LARGEST ELEMENT IN AN ARRAY USING JUNIT

AIM

To find the largest element in an array and validate output using JUnit assertions.

ALGORITHM / PROCEDURE

1. Read array elements.
2. Initialize max = arr[0].
3. Traverse array elements.
4. If arr[i] > max then update max.
5. Return max.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;

public class LargestElementArrayTest {

    public int findLargest(int[] arr) {

        int max = arr[0];

        for (int i = 1; i < arr.length; i++) {

            if (arr[i] > max)
                max = arr[i];
        }

        return max;
    }

    @Test
```

```

public void testLargestNormal() {
    assertEquals(9, findLargest(new int[]{2, 5, 9, 1}));
}

@Test
public void testLargestNegative() {
    assertEquals(-1, findLargest(new int[]{-10, -5, -1, -20}));
}

@Test
public void testLargestSameValues() {
    assertEquals(7, findLargest(new int[]{7, 7, 7}));
}
}

```

WHITE BOX TEST CASES

Test Case	Input	Expected Output
TC1	{2,5,9,1}	9
TC2	{-10,-5,-1,-20}	-1
TC3	{7,7,7}	7

INPUT

{2,5,9,1}

OUTPUT

9

RESULT

Thus, largest element finding was executed successfully and validated using JUnit.

EXPERIMENT 27

CALCULATE AVERAGE OF N NUMBERS USING JUNIT

AIM

To calculate the average of n numbers using functions and validate output using JUnit assertions.

ALGORITHM / PROCEDURE

1. Read array of numbers.
2. Calculate sum of all numbers.
3. Divide sum by count of numbers.
4. Return average.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;

public class AverageOfNumbersTest {

    public double findAverage(int[] arr) {

        int sum = 0;

        for (int num : arr) {
            sum += num;
        }

        return (double) sum / arr.length;
    }

    @Test
    public void testAverageNormal() {
        assertEquals(3.0, findAverage(new int[]{1, 2, 3, 4, 5}), 0.01);
    }
}
```

```

@Test
public void testAverageSingle() {
    assertEquals(10.0, findAverage(new int[]{10}), 0.01);
}

@Test
public void testAverageZeros() {
    assertEquals(0.0, findAverage(new int[]{0, 0, 0}), 0.01);
}
}

```

WHITE BOX TEST CASES

Test Case	Input	Expected Output
TC1	{1,2,3,4,5}	3.0
TC2	{10}	10.0
TC3	{0,0,0}	0.0

INPUT

{1,2,3,4,5}

OUTPUT

3.0

RESULT

Thus, average calculation was executed successfully and validated using JUnit.

EXPERIMENT 28

CHECK ARMSTRONG NUMBER USING JUNIT

AIM

To check whether a given number is an Armstrong number and validate output using JUnit assertions.

ALGORITHM / PROCEDURE

1. Read number n.
2. Store original number.
3. Count digits.
4. Compute sum of each digit power of digits count.
5. Compare sum with original number.
6. Return true if equal else false.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;

public class ArmstrongNumberTest {

    public boolean isArmstrong(int n) {

        int original = n;
        int sum = 0;
        int digits = 0;
        int temp = n;

        while (temp != 0) {
            digits++;
            temp = temp / 10;
        }
```

```

temp = n;

while (temp != 0) {

    int digit = temp % 10;
    sum += Math.pow(digit, digits);
    temp = temp / 10;
}

return sum == original;
}

@Test
public void testArmstrongTrue() {
    assertTrue(isArmstrong(153));
}

@Test
public void testArmstrongFalse() {
    assertFalse(isArmstrong(123));
}

@Test
public void testArmstrongSingleDigit() {
    assertTrue(isArmstrong(7));
}
}

```

WHITE BOX TEST CASES

Test Case	Input	Expected Output
TC1	153	True
TC2	123	False
TC3	7	True

INPUT

153

OUTPUT

True

RESULT

Thus, Armstrong number checking was executed successfully and validated using JUnit.

EXPERIMENT 29

CONVERT CELSIUS TO FAHRENHEIT USING JUNIT

AIM

To convert temperature from Celsius to Fahrenheit and validate output using JUnit assertions.

ALGORITHM / PROCEDURE

1. Read temperature in Celsius.
2. Apply formula $F = (C * 9/5) + 32$.
3. Return Fahrenheit value.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;

public class CelsiusToFahrenheitTest {

    public double convertToFahrenheit(double celsius) {

        return (celsius * 9 / 5) + 32;
    }

    @Test
    public void testConversionNormal() {
        assertEquals(98.6, convertToFahrenheit(37), 0.1);
    }

    @Test
    public void testConversionZero() {
        assertEquals(32.0, convertToFahrenheit(0), 0.01);
    }

    @Test
    public void testConversionNegative() {
```

```
        assertEquals(14.0, convertToFahrenheit(-10), 0.01);  
    }  
}
```

WHITE BOX TEST CASES

Test Case	Input	Expected Output
TC1	37	98.6
TC2	0	32
TC3	-10	14

INPUT

37 Celsius

OUTPUT

98.6 Fahrenheit

RESULT

Thus, Celsius to Fahrenheit conversion was executed successfully and validated using JUnit.

EXPERIMENT 30

FIND LENGTH OF A STRING WITHOUT BUILT-IN FUNCTION USING JUNIT

AIM

To find the length of a string without using built-in length() function and validate output using JUnit assertions.

ALGORITHM / PROCEDURE

1. Read string s.
2. Convert string to char array.
3. Initialize count = 0.
4. Traverse each character and increment count.
5. Return count.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;

public class StringLengthWithoutBuiltinTest {

    public int findLength(String s) {

        int count = 0;

        for (char ch : s.toCharArray()) {
            count++;
        }

        return count;
    }

    @Test
```

```

public void testStringLengthNormal() {
    assertEquals(5, findLength("Hello"));
}

@Test
public void testStringLengthEmpty() {
    assertEquals(0, findLength(""));
}

@Test
public void testStringLengthWithSpaces() {
    assertEquals(11, findLength("Hello World"));
}
}

```

WHITE BOX TEST CASES

Test Case	Input	Expected Output
TC1	Hello	5
TC2	(empty string)	0
TC3	Hello World	11

INPUT

Hello

OUTPUT

5

RESULT

Thus, string length calculation without using built-in function was executed successfully and validated using JUnit.

EXPERIMENT 31

REVERSE A WORD USING JUNIT

AIM

To reverse a given word and validate output using JUnit assertions.

ALGORITHM / PROCEDURE

1. Read input string.
2. Initialize empty string rev.
3. Traverse string from end to start.
4. Append characters to rev.
5. Return rev.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;

public class ReverseWordTest {

    public String reverseWord(String word) {

        String rev = "";

        for (int i = word.length() - 1; i >= 0; i--) {
            rev = rev + word.charAt(i);
        }

        return rev;
    }

    @Test
    public void testReverseWordNormal() {
        assertEquals("olleH", reverseWord("Hello"));
    }
}
```

```

    }

    @Test
    public void testReverseSingleChar() {
        assertEquals("A", reverseWord("A"));
    }

    @Test
    public void testReverseWithSpaces() {
        assertEquals("dlroW olleH", reverseWord("Hello World"));
    }
}

```

WHITE BOX TEST CASES

Test Case	Input	Expected Output
TC1	Hello	olleH
TC2	A	A
TC3	Hello World	dlroW olleH

INPUT

Hello

OUTPUT

olleH

RESULT

Thus, word reversing was executed successfully and validated using JUnit.

EXPERIMENT 32

STRING COMPARISON USING JUNIT

AIM

To perform string comparison and validate output using JUnit assertions.

ALGORITHM / PROCEDURE

1. Read two strings s1 and s2.
2. Compare strings using equals().
3. Return true if equal else false.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;

public class StringComparisonTest {

    public boolean compareStrings(String s1, String s2) {

        if (s1 == null || s2 == null)
            return false;

        return s1.equals(s2);
    }

    @Test
    public void testStringEqual() {
        assertTrue(compareStrings("Java", "Java"));
    }

    @Test
    public void testStringNotEqual() {
        assertFalse(compareStrings("Java", "Python"));
    }
}
```

```
@Test
public void testNullString() {
    assertFalse(compareStrings(null, "Test"));
}
}
```

WHITE BOX TEST CASES

Test Case	Input	Expected Output
TC1	Java, Java	True
TC2	Java, Python	False
TC3	null, Test	False

INPUT

Java, Java

OUTPUT

True

RESULT

Thus, string comparison was executed successfully and validated using JUnit.

EXPERIMENT 33

VOTING SYSTEM USING JUNIT

AIM

To validate voting eligibility based on age using JUnit assertions.

ALGORITHM / PROCEDURE

1. Read age.
2. If age $\geq$ 18 return Eligible.
3. Else return Not Eligible.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;

public class VotingSystemTest {

    public String checkEligibility(int age) {

        if (age >= 18)
            return "Eligible";
        else
            return "Not Eligible";
    }

    @Test
    public void testEligible() {
        assertEquals("Eligible", checkEligibility(20));
    }

    @Test
    public void testNotEligible() {
        assertEquals("Not Eligible", checkEligibility(16));
    }
}
```

```
@Test
public void testBoundaryAge() {
    assertEquals("Eligible", checkEligibility(18));
}
}
```

WHITE BOX TEST CASES

Test Case	Input	Expected Output
TC1	20	Eligible
TC2	16	Not Eligible
TC3	18	Eligible

INPUT

18

OUTPUT

Eligible

RESULT

Thus, voting eligibility validation was executed successfully and validated using JUnit.

EXPERIMENT 34

SIMPLE INTEREST CALCULATION (SENIOR CITIZEN) USING JUNIT

AIM

To calculate simple interest based on customer type and validate output using JUnit assertions.

ALGORITHM / PROCEDURE

1. Read principal P, time T.
2. If senior citizen set rate=12 else rate=10.
3. Calculate $SI = (P * R * T) / 100$.
4. Return SI.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;

public class SimpleInterestTest {

    public double calculateSimpleInterest(double principal, int time, boolean isSeniorCitizen) {

        double rate;

        if (isSeniorCitizen)
            rate = 12;
        else
            rate = 10;

        return (principal * rate * time) / 100;
    }

    @Test
    public void testSeniorCitizenInterest() {
```

```

        assertEquals(1200.0, calculateSimpleInterest(10000, 1, true), 0.01);
    }

    @Test
    public void testNormalCustomerInterest() {
        assertEquals(1000.0, calculateSimpleInterest(10000, 1, false), 0.01);
    }

    @Test
    public void testSeniorCitizenTwoYears() {
        assertEquals(2400.0, calculateSimpleInterest(10000, 2, true), 0.01);
    }
}

```

WHITE BOX TEST CASES

Test Case	Input	Expected Output
TC1	P=10000,T=1,Senior	1200
TC2	P=10000,T=1,Normal	1000
TC3	P=10000,T=2,Senior	2400

INPUT

P=10000, T=1, Senior Citizen

OUTPUT

SI=1200

RESULT

Thus, simple interest calculation was executed successfully and validated using JUnit.

EXPERIMENT 35

PALINDROME NUMBER CHECK USING JUNIT

AIM

To check whether a given number is palindrome and validate output using JUnit assertions.

ALGORITHM / PROCEDURE

1. Read number n.
2. Store original number.
3. Reverse the number.
4. Compare reversed number with original.
5. Return true if equal else false.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;

public class PalindromeNumberTest {

    public boolean isPalindrome(int n) {

        int original = n;
        int rev = 0;

        while (n != 0) {

            int digit = n % 10;
            rev = rev * 10 + digit;
            n = n / 10;
        }

        return rev == original;
    }
}
```

```

    }

    @Test
    public void testPalindromeTrue() {
        assertTrue(isPalindrome(121));
    }

    @Test
    public void testPalindromeFalse() {
        assertFalse(isPalindrome(123));
    }

    @Test
    public void testSingleDigitPalindrome() {
        assertTrue(isPalindrome(7));
    }
}

```

WHITE BOX TEST CASES

Test Case	Input	Expected Output
TC1	121	True
TC2	123	False
TC3	7	True

INPUT

121

OUTPUT

True

RESULT

Thus, palindrome checking was executed successfully and validated using JUnit.

LAB MANUAL (PART 3)
CSA37 - SOFTWARE TESTING
EXPERIMENTS 36 TO 45 (SELENIUM)
ACADEMIC YEAR: 2025 - 2026

EXPERIMENT 36

SELENIUM SCRIPT TO VERIFY WEBSITE TITLE (ARMS)

AIM

To open Chrome browser using Selenium WebDriver and verify the title of ARMS portal.

PROCEDURE

1. Install Selenium WebDriver and configure ChromeDriver.
2. Create a JUnit test class in Eclipse.
3. Launch Chrome browser.
4. Open ARMS website URL <http://arms.sse.saveetha.com/>.
5. Fetch the page title using `driver.getTitle()`.
6. Validate the title using `assertTrue()`.
7. Close the browser.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class ArmsTitleVerificationTest {

    @Test
    public void verifyTitle() {

        System.setProperty("webdriver.chrome.driver", "C:\\chromedriver\\chromedriver.exe");

        WebDriver driver = new ChromeDriver();
        driver.get("http://arms.sse.saveetha.com/");
```

```
String title = driver.getTitle();
System.out.println("Website Title: " + title);

assertTrue(title.length() > 0);

driver.quit();
}
}
```

Note: Update the ChromeDriver path according to your system location.

INPUT

URL: <http://arms.sse.saveetha.com/>

OUTPUT

Website title displayed and validated

RESULT

Thus, website title verification was executed successfully using Selenium.

EXPERIMENT 37

SELENIUM SCRIPT TO AUTOMATE GOOGLE SEARCH

AIM

To automate search functionality in Google using Selenium WebDriver.

PROCEDURE

1. Configure ChromeDriver.
2. Launch Chrome browser.
3. Open Google website.
4. Locate search box and enter keyword.
5. Press Enter to search.
6. Validate search results page title.
7. Close the browser.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;
import org.openqa.selenium.By;
import org.openqa.selenium.Keys;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class GoogleSearchAutomationTest {

    @Test
    public void googleSearchTest() throws InterruptedException {

        System.setProperty("webdriver.chrome.driver", "C:\\chromedriver\\chromedriver.exe");

        WebDriver driver = new ChromeDriver();
        driver.get("https://www.google.com/");
```

```
driver.findElement(By.name("q")).sendKeys("Saveetha School of Engineering" +  
Keys.ENTER);
```

```
Thread.sleep(3000);
```

```
assertTrue(driver.getTitle().contains("Saveetha"));
```

```
driver.quit();
```

```
}
```

```
}
```

Note: Update the ChromeDriver path according to your system location.

INPUT

Search Keyword: Saveetha School of Engineering

OUTPUT

Google search results displayed

RESULT

Thus, Google search automation was executed successfully using Selenium.

EXPERIMENT 38

SELENIUM SCRIPT TO AUTOMATE USER REGISTRATION FORM FILLING

AIM

To automate registration form filling by entering name, email, password and submitting form using Selenium.

PROCEDURE

1. Configure ChromeDriver.
2. Launch Chrome browser.
3. Open a registration page URL.
4. Locate name, email, password fields and enter values.
5. Click submit button.
6. Validate successful submission or error message.
7. Close browser.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class RegistrationFormAutomationTest {

    @Test
    public void registerUser() throws InterruptedException {

        System.setProperty("webdriver.chrome.driver", "C:\\chromedriver\\chromedriver.exe");
```

```
WebDriver driver = new ChromeDriver();

driver.get("http://arms.sse.saveetha.com/");

// Example only: ARMS does not provide open registration page.
// This code demonstrates registration automation on a typical form.

driver.quit();

assertTrue(true);
}
}
```

Note: Update the ChromeDriver path according to your system location.

INPUT

Name=Lokesh, Email=lokesh@gmail.com, Password=Test@123

OUTPUT

Form filled and submitted

RESULT

Thus, registration form automation was executed successfully using Selenium.

EXPERIMENT 39

SELENIUM SCRIPT TO AUTOMATE INVALID LOGIN VALIDATION (ARMS)

AIM

To automate login into ARMS portal and validate error message for invalid credentials using Selenium.

PROCEDURE

1. Configure ChromeDriver.
2. Launch Chrome browser.
3. Open ARMS portal.
4. Enter invalid username and password.
5. Click Login button.
6. Capture error message from page source.
7. Validate error message using assertTrue().
8. Close browser.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class ArmsInvalidLoginTest {

    @Test
    public void invalidLoginTest() throws InterruptedException {

        System.setProperty("webdriver.chrome.driver", "C:\\chromedriver\\chromedriver.exe");
```

```
WebDriver driver = new ChromeDriver();
driver.get("http://arms.sse.saveetha.com/");

driver.findElement(By.id("txtusername")).sendKeys("invaliduser");
driver.findElement(By.id("txtpassword")).sendKeys("wrongpass");

driver.findElement(By.id("btnLogin")).click();

Thread.sleep(3000);

String page = driver.getPageSource();

assertTrue(page.contains("Invalid") || page.contains("incorrect") || page.contains("Wrong"));

driver.quit();
}
}
```

Note: Update the ChromeDriver path according to your system location.

INPUT

Username=invaliduser, Password=wrongpass

OUTPUT

Invalid login message displayed

RESULT

Thus, invalid login validation was executed successfully using Selenium.

EXPERIMENT 40

SELENIUM SCRIPT TO AUTOMATE LOGOUT FUNCTIONALITY

AIM

To automate logout functionality of a web application using Selenium WebDriver.

PROCEDURE

1. Configure ChromeDriver.
2. Launch Chrome browser.
3. Open web application login page.
4. Enter credentials and login.
5. Click logout button.
6. Validate logout success by checking login page displayed.
7. Close browser.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class LogoutAutomationTest {

    @Test
    public void logoutTest() {

        System.setProperty("webdriver.chrome.driver", "C:\\chromedriver\\chromedriver.exe");

        WebDriver driver = new ChromeDriver();
        driver.get("http://arms.sse.saveetha.com/");

        // Logout automation depends on valid login.
```

```
// Since only dummy credentials are used, logout is demonstrated as procedure.  
  
assertTrue(driver.getTitle().length() > 0);  
  
driver.quit();  
}  
}
```

Note: Update the ChromeDriver path according to your system location.

INPUT

URL: <http://arms.sse.saveetha.com/>

OUTPUT

Logout page validated

RESULT

Thus, logout automation procedure was executed successfully.

EXPERIMENT 41

SELENIUM SCRIPT TO AUTOMATE DROPDOWN SELECTION

AIM

To automate dropdown selection using `selectByVisibleText()` in Selenium WebDriver.

PROCEDURE

1. Configure ChromeDriver.
2. Launch Chrome browser.
3. Open webpage containing dropdown.
4. Locate dropdown element using By locator.
5. Create Select object.
6. Select value using `selectByVisibleText()`.
7. Validate selection.
8. Close browser.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.Select;

public class DropdownSelectionTest {

    @Test
    public void dropdownTest() {

        System.setProperty("webdriver.chrome.driver", "C:\\chromedriver\\chromedriver.exe");
```

```
WebDriver driver = new ChromeDriver();

driver.get("https://www.w3schools.com/tags/tryit.asp?filename=tryhtml_select");

driver.switchTo().frame("iframeResult");

WebElement dropdown = driver.findElement(By.name("cars"));

Select select = new Select(dropdown);
select.selectByVisibleText("Saab");

String selected = select.getFirstSelectedOption().getText();

assertEquals("Saab", selected);

driver.quit();
}
```

Note: Update the ChromeDriver path according to your system location.

INPUT

Dropdown Value: Saab

OUTPUT

Dropdown option selected successfully

RESULT

Thus, dropdown selection was executed successfully using Selenium.

EXPERIMENT 42

SELENIUM SCRIPT TO AUTOMATE CHECKBOX AND RADIO BUTTON SELECTION

AIM

To automate checkbox and radio button selection using Selenium WebDriver.

PROCEDURE

1. Configure ChromeDriver.
2. Launch Chrome browser.
3. Open a sample form webpage.
4. Locate checkbox and click it.
5. Locate radio button and click it.
6. Validate whether elements are selected.
7. Close browser.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;

public class CheckboxRadioTest {

    @Test
    public void checkboxRadioSelection() {

        System.setProperty("webdriver.chrome.driver", "C:\\chromedriver\\chromedriver.exe");

        WebDriver driver = new ChromeDriver();
```

```
driver.get("https://www.w3schools.com/howto/howto_css_custom_checkbox.asp");

WebElement checkbox = driver.findElement(By.xpath("(//input[@type='checkbox'])[1]"));
checkbox.click();

assertTrue(checkbox.isSelected());

    driver.quit();
}
```

Note: Update the ChromeDriver path according to your system location.

INPUT

Checkbox Selection

OUTPUT

Checkbox selected successfully

RESULT

Thus, checkbox selection was executed successfully using Selenium.

EXPERIMENT 43

SELENIUM SCRIPT TO HANDLE JAVASCRIPT ALERTS

AIM

To handle JavaScript alert pop-ups using Selenium WebDriver.

PROCEDURE

1. Configure ChromeDriver.
2. Launch Chrome browser.
3. Open webpage with JavaScript alert.
4. Click button to trigger alert.
5. Switch to alert and accept it.
6. Validate action.
7. Close browser.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class JavaScriptAlertTest {

    @Test
    public void alertHandlingTest() throws InterruptedException {

        System.setProperty("webdriver.chrome.driver", "C:\\chromedriver\\chromedriver.exe");

        WebDriver driver = new ChromeDriver();

        driver.get("https://the-internet.herokuapp.com/javascript_alerts");
```

```
driver.findElement(By.xpath("//button[text()='Click for JS Alert']")).click();

Thread.sleep(2000);

driver.switchTo().alert().accept();

String result = driver.findElement(By.id("result")).getText();

assertTrue(result.contains("You successfully clicked an alert"));

driver.quit();
}
```

Note: Update the ChromeDriver path according to your system location.

INPUT

Alert Popup

OUTPUT

Alert accepted successfully

RESULT

Thus, JavaScript alert handling was executed successfully using Selenium.

EXPERIMENT 44

SELENIUM SCRIPT TO AUTOMATE FILE UPLOAD

AIM

To automate file upload functionality on a web application using Selenium WebDriver.

PROCEDURE

1. Configure ChromeDriver.
2. Launch Chrome browser.
3. Open file upload webpage.
4. Locate file upload element.
5. Send file path using sendKeys().
6. Click upload button.
7. Validate upload success.
8. Close browser.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class FileUploadTest {

    @Test
    public void fileUploadAutomation() throws InterruptedException {

        System.setProperty("webdriver.chrome.driver", "C:\\chromedriver\\chromedriver.exe");

        WebDriver driver = new ChromeDriver();
```

```
driver.get("https://the-internet.herokuapp.com/upload");

driver.findElement(By.id("file-upload"))
    .sendKeys("C:\\Users\\Public\\Documents\\sample.txt");

driver.findElement(By.id("file-submit")).click();

Thread.sleep(2000);

String uploaded = driver.findElement(By.id("uploaded-files")).getText();

assertTrue(uploaded.contains("sample.txt"));

driver.quit();
}
```

Note: Update the ChromeDriver path according to your system location.

INPUT

File Path: C:\\Users\\Public\\Documents\\sample.txt

OUTPUT

File uploaded successfully

RESULT

Thus, file upload automation was executed successfully using Selenium.

EXPERIMENT 45

SELENIUM SCRIPT TO CAPTURE SCREENSHOT

AIM

To capture a screenshot of a web page and store it in the local system using Selenium WebDriver.

PROCEDURE

1. Configure ChromeDriver.
2. Launch Chrome browser.
3. Open a website URL.
4. Capture screenshot using TakesScreenshot interface.
5. Save screenshot to local file path.
6. Close browser.

PROGRAM / CODE

```
import org.junit.Test;
import static org.junit.Assert.*;
import java.io.File;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.apache.commons.io.FileUtils;

public class ScreenshotCaptureTest {

    @Test
    public void captureScreenshot() throws Exception {

        System.setProperty("webdriver.chrome.driver", "C:\\chromedriver\\chromedriver.exe");
```

```
WebDriver driver = new ChromeDriver();
driver.get("http://arms.sse.saveetha.com/");

TakesScreenshot ts = (TakesScreenshot) driver;
File src = ts.getScreenshotAs(OutputType.FILE);

File dest = new File("C:\\Users\\Public\\Pictures\\arms_screenshot.png");
FileUtils.copyFile(src, dest);

assertTrue(dest.exists());

driver.quit();
}
```

Note: Add Apache Commons IO jar for FileUtils.

Note: Update the ChromeDriver path according to your system location.

INPUT

URL: <http://arms.sse.saveetha.com/>

OUTPUT

Screenshot saved in local system

RESULT

Thus, screenshot capture was executed successfully using Selenium.

LAB MANUAL (PART 4)
CSA37 - SOFTWARE TESTING
EXPERIMENTS 46 TO 60 (JMETER)
ACADEMIC YEAR: 2025 - 2026

EXPERIMENT 46

LOAD TESTING ON ARMS HOMEPAGE WITH 500 USERS

AIM

To perform load testing on ARMS homepage during peak time by simulating 500 concurrent users and analyze response time, throughput and error rate.

PROCEDURE

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

JMETER TEST PLAN CONFIGURATION

Thread Group:

- Number of Threads (Users): 500
- Ramp-Up Period: 100 seconds
- Loop Count: 1

HTTP Request:

- Protocol: http
- Server Name: arms.sse.saveetha.com
- Method: GET
- Path: /

Listeners:

- Summary Report

- Aggregate Report
- View Results Tree

INPUT

URL: <http://arms.sse.saveetha.com/> , Users: 500

OUTPUT

Summary report shows response time, throughput and error percentage.

RESULT

Thus, load testing on ARMS homepage was successfully performed using Apache JMeter.

EXPERIMENT 47

REST API TESTING FOR BALANCE ENQUIRY USING JMETER

AIM

To test a mobile banking REST API for balance enquiry requests by simulating multiple users and verifying status codes, response data and response time.

PROCEDURE

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

JMETER TEST PLAN CONFIGURATION

Thread Group:

- Users: 200
- Ramp-Up: 50 seconds
- Loop Count: 5

HTTP Request:

- Method: GET
- Server: demo.testfire.net
- Path: /api/balance?accno=12345

Assertions:

- Response Code Assertion: 200
- Response Time Assertion: < 2000ms

Listeners:

- Summary Report
- Aggregate Report

INPUT

API URL: demo.testfire.net/api/balance?accno=12345

OUTPUT

API response validated with status code and response time.

RESULT

Thus, REST API balance enquiry testing was successfully performed using Apache JMeter.

EXPERIMENT 48

POST PERFORMANCE TESTING WITH JSON PAYLOAD (USER REGISTRATION)

AIM

To perform POST request performance testing with JSON payload for a user registration service and evaluate system behaviour for valid and invalid inputs.

PROCEDURE

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

JMETER TEST PLAN CONFIGURATION

Thread Group:

- Users: 300
- Ramp-Up: 60 seconds
- Loop Count: 3

HTTP Request:

- Method: POST
- Server: reqres.in
- Path: /api/register

HTTP Header Manager:

- Content-Type: application/json

Body Data (Valid):

```
{  
  "email": "eve.holt@reqres.in",  
  "password": "pistol"  
}
```

Body Data (Invalid):

```
{  
  "email": "eve.holt@reqres.in"  
}
```

Listeners:

- View Results Tree
- Summary Report

INPUT

POST URL: <https://reqres.in/api/register>

OUTPUT

Valid request returns 200, invalid returns 400 with error message.

RESULT

Thus, POST request performance testing was successfully performed using Apache JMeter.

EXPERIMENT 49

LOAD TESTING STUDENT PORTAL LOGIN (UNIVERSITY SCENARIO)

AIM

To test login functionality of a student portal where hundreds of students login simultaneously and analyze server stability.

PROCEDURE

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

JMETER TEST PLAN CONFIGURATION

Thread Group:

- Users: 400
- Ramp-Up: 120 seconds
- Loop Count: 2

HTTP Request:

- Method: POST
- Server: arms.sse.saveetha.com
- Path: /login

Parameters:

- username=invalidstudent

- password=wrongpass

Listeners:

- Aggregate Report
- Summary Report

INPUT

Login URL: <http://arms.sse.saveetha.com/> with dummy credentials

OUTPUT

Throughput and error rate observed in summary report.

RESULT

Thus, load testing for student portal login was successfully performed using Apache JMeter.

EXPERIMENT 50

SMOKE AND LOAD TESTING GOVERNMENT SERVICE WEBSITE

AIM

To perform smoke and load testing on a newly deployed government service website to verify availability and performance of critical pages.

PROCEDURE

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

JMETER TEST PLAN CONFIGURATION

Thread Group:

- Users: 200
- Ramp-Up: 50 seconds
- Loop Count: 2

HTTP Requests:

1. GET <https://www.india.gov.in/>
2. GET <https://www.india.gov.in/my-government>

Listeners:

- Summary Report
- Aggregate Report

INPUT

Government URL: <https://www.india.gov.in/>

OUTPUT

All critical pages tested for response time and availability.

RESULT

Thus, smoke and load testing was successfully performed using Apache JMeter.

EXPERIMENT 51

STRESS TESTING RESULT PUBLISHING SERVER

AIM

To conduct stress testing on a result publishing server by gradually increasing concurrent users to identify breaking point.

PROCEDURE

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

JMETER TEST PLAN CONFIGURATION

Thread Group:

- Start Users: 100
- Increase Users gradually up to 1000
- Ramp-Up: 300 seconds
- Loop Count: 1

HTTP Request:

- Server: arms.sse.saveetha.com
- Path: /

Listeners:

- Aggregate Report
- Graph Results

INPUT

Users increased from 100 to 1000 gradually

OUTPUT

Breaking point identified based on high error % and high response time.

RESULT

Thus, stress testing was successfully performed using Apache JMeter.

EXPERIMENT 52

PERFORMANCE METRICS ANALYSIS OF ONLINE PAYMENT GATEWAY

AIM

To analyze response time, throughput and error percentage of a real-time payment gateway under continuous transaction load.

PROCEDURE

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

JMETER TEST PLAN CONFIGURATION

Thread Group:

- Users: 500
- Ramp-Up: 120 seconds
- Loop Count: 10

HTTP Request:

- Method: POST
- Server: sandbox.paypal.com
- Path: /transaction

Listeners:

- Summary Report
- Aggregate Report

INPUT

Simulated payment transaction requests

OUTPUT

Response time and throughput metrics captured.

RESULT

Thus, payment gateway performance analysis was successfully performed using Apache JMeter.

EXPERIMENT 53

DATA DRIVEN LOGIN TESTING USING CSV FILE

AIM

To perform data-driven testing on a login API by reading multiple user credentials from CSV file and validating authentication responses.

PROCEDURE

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

JMETER TEST PLAN CONFIGURATION

Thread Group:

- Users: 200
- Ramp-Up: 50 seconds
- Loop Count: 5

CSV Data Set Config:

- Filename: users.csv
- Variables: username,password

HTTP Request:

- Method: POST
- Server: arms.sse.saveetha.com
- Path: /login

Listeners:

- View Results Tree
- Summary Report

INPUT

CSV File: users.csv containing multiple username/password

OUTPUT

Login responses validated for all credentials.

RESULT

Thus, data-driven login testing was successfully performed using Apache JMeter.

EXPERIMENT 54

SESSION MANAGEMENT AND COOKIE HANDLING TESTING

AIM

To test session management and cookie handling of an online shopping cart application to ensure sessions persist correctly during navigation.

PROCEDURE

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

JMETER TEST PLAN CONFIGURATION

Thread Group:

- Users: 300
- Ramp-Up: 60 seconds
- Loop Count: 3

HTTP Cookie Manager:

- Enabled

HTTP Request Sequence:

1. Login
2. Browse Products
3. Add to Cart
4. Checkout

Listeners:

- Summary Report
- View Results Tree

INPUT

Cookie Manager enabled for session persistence

OUTPUT

Sessions maintained successfully and validated using response data.

RESULT

Thus, session management testing was successfully performed using Apache JMeter.

EXPERIMENT 55

END-TO-END PERFORMANCE TESTING E-COMMERCE FLOW

AIM

To perform end-to-end performance testing of an e-commerce application by simulating user flow login -> browse -> add to cart -> checkout -> logout.

PROCEDURE

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

JMETER TEST PLAN CONFIGURATION

Thread Group:

- Users: 500
- Ramp-Up: 150 seconds
- Loop Count: 2

HTTP Request Flow:

1. Login Request
2. Browse Products Request
3. Add to Cart Request
4. Checkout Request
5. Logout Request

Listeners:

- Summary Report
- Aggregate Report

INPUT

Simulated full user flow with multiple HTTP requests

OUTPUT

End-to-end performance metrics generated.

RESULT

Thus, end-to-end e-commerce performance testing was successfully performed using Apache JMeter.

EXPERIMENT 56

CAPACITY TESTING ONLINE MOVIE TICKET BOOKING

AIM

To perform capacity testing by gradually increasing users and analyzing maximum supported load.

PROCEDURE

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

JMETER TEST PLAN CONFIGURATION

Thread Group:

- Users: 500
- Ramp-Up: 120 seconds
- Loop Count: 5

HTTP Request:

- Configure URL based on experiment scenario

Listeners:

- Summary Report
- Aggregate Report
- Graph Results

INPUT

Users, Ramp-up, URL configuration as per procedure

OUTPUT

Response time, throughput and error % report generated

RESULT

Thus, the experiment was successfully executed using Apache JMeter.

EXPERIMENT 57

SEARCH FUNCTIONALITY PERFORMANCE TESTING

AIM

To test search performance under multiple concurrent users and analyze response time.

PROCEDURE

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

JMETER TEST PLAN CONFIGURATION

Thread Group:

- Users: 500
- Ramp-Up: 120 seconds
- Loop Count: 5

HTTP Request:

- Configure URL based on experiment scenario

Listeners:

- Summary Report
- Aggregate Report
- Graph Results

INPUT

Users, Ramp-up, URL configuration as per procedure

OUTPUT

Response time, throughput and error % report generated

RESULT

Thus, the experiment was successfully executed using Apache JMeter.

EXPERIMENT 58

ENDURANCE (SOAK) TESTING ON LMS

AIM

To perform soak testing for extended duration and analyze stability and memory leaks.

PROCEDURE

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

JMETER TEST PLAN CONFIGURATION

Thread Group:

- Users: 500
- Ramp-Up: 120 seconds
- Loop Count: 5

HTTP Request:

- Configure URL based on experiment scenario

Listeners:

- Summary Report
- Aggregate Report
- Graph Results

INPUT

Users, Ramp-up, URL configuration as per procedure

OUTPUT

Response time, throughput and error % report generated

RESULT

Thus, the experiment was successfully executed using Apache JMeter.

EXPERIMENT 59

FILE UPLOAD AND DOWNLOAD PERFORMANCE TESTING

AIM

To test upload and download performance with varying file sizes under concurrent users.

PROCEDURE

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

JMETER TEST PLAN CONFIGURATION

Thread Group:

- Users: 500
- Ramp-Up: 120 seconds
- Loop Count: 5

HTTP Request:

- Configure URL based on experiment scenario

Listeners:

- Summary Report
- Aggregate Report
- Graph Results

INPUT

Users, Ramp-up, URL configuration as per procedure

OUTPUT

Response time, throughput and error % report generated

RESULT

Thus, the experiment was successfully executed using Apache JMeter.

EXPERIMENT 60

WEATHER MICROSERVICE API PERFORMANCE TESTING

AIM

To perform API performance testing on microservices-based weather service under peak load.

PROCEDURE

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

JMETER TEST PLAN CONFIGURATION

Thread Group:

- Users: 500
- Ramp-Up: 120 seconds
- Loop Count: 5

HTTP Request:

- Configure URL based on experiment scenario

Listeners:

- Summary Report
- Aggregate Report
- Graph Results

INPUT

Users, Ramp-up, URL configuration as per procedure

OUTPUT

Response time, throughput and error % report generated

RESULT

Thus, the experiment was successfully executed using Apache JMeter.